

SMC Scratchpad RAM — Test Plan

SMC Scratchpad RAM — Test Plan

Contents

1. Strategy
2. Test benches
3. Primary bench test list
4. Negative bench test list
5. Coverage goals & results
6. Running the tests
7. Traceability to the specification

SMC Scratchpad RAM — Test Plan

Document: 03_SCRATCHPAD_RAM_Test_Plan.md **Module:** smc::scratchpad_ram **Companion docs:** 01_SCRATCHPAD_RAM_Specification.md, 02_SCRATCHPAD_RAM_LowLevel_Design.md

Contents

- 1. Strategy
- 2. Test benches
- 3. Primary bench test list
- 4. Negative bench test list
- 5. Coverage goals & results
- 6. Running the tests
- 7. Traceability to the specification

1. Strategy

The scratchpad is verified by two self-checking SystemC test benches that drive the DUT through its TLM-2.0 target socket and assert on response status and data. The benches print `ALL TESTS PASSED` on success and a non-zero exit code on failure; CTest matches on those patterns.

The split mirrors the Boot ROM suite:

- A **primary bench** drives the normal data path (reads, writes, byte-enables, ECC scrub, CCI) on a fully-constructed DUT.
- A **negative bench** exercises the destructive constructor paths (`SC_REPORT_FATAL`) using a `SC_THROW` report handler, plus edge cases that the primary path does not reach.

2. Test benches

Bench	Binary	Focus
scratchpad_ram_tb.cpp	scratchpad_ram_tb	Functional data path, byte-enables, ECC scrub, reset retention, CCI.
scratchpad_ram_neg_tb.cpp	scratchpad_ram_neg_tb	Constructor fatals, preload errors, uncorrectable-ECC read, parser edges.

Both link `libsmc_scratchpad_ram.a` + SystemC + CCI and register with CTest.

3. Primary bench test list

#	Test	Asserts
1	Hex preload	First 4 words read back == fixture (<code>b_transport</code> and <code>dbg_read64</code>).
2	Read-after-write (1/2/4/8 B)	A write is observable on the next read.
3	Sub-word reads	1/2/4/8 B reads are coherent with the 64-bit word.
4	Byte-enabled write	Only enabled bytes change (<code>0x00FF00FF00FF00FF</code> pattern).
5	Reset retention	Contents survive a reset pulse (SRAM, not ROM).
6	Zero-init region	Unwritten high address reads as zero.
7	<code>transport_dbg</code>	Back-door write then read round-trips.
8	<code>dbg_load_bytes</code>	Loaded bytes read back through <code>b_transport</code> ; OOB load returns 0.
9	SECCDED ECC	Correctable error scrubbed (read OK); uncorrectable read → <code>TLM_GENERIC_ERROR_RESPONSE</code> ; write clears the error.
10	Negative paths	Out-of-window, misaligned, bad width, streaming mismatch, bad command.
11	CCI	Param discovery, <code>access_delay_ns</code> mutation, <code>size_bytes</code> immutability.
12	<code>dump_state</code>	Output contains the expected fields.
13	Annotated delay	<code>b_transport</code> accumulates a non-zero delay.

4. Negative bench test list

#	Test	Asserts
1	<code>size_bytes == 0</code>	<code>SC_REPORT_FATAL</code> .
2	<code>size_bytes % 8 != 0</code>	<code>SC_REPORT_FATAL</code> .
3	<code>access_delay_ns < 0</code>	<code>SC_REPORT_FATAL</code> .
4	Missing hex preload	<code>SC_REPORT_FATAL</code> .
5	Missing binary preload	<code>SC_REPORT_FATAL</code> .
6	Empty <code>.img</code> preload	<code>SC_REPORT_FATAL</code> .
7	Malformed hex (non-hex chars)	<code>SC_REPORT_FATAL</code> .
8	Hex with trailing junk	<code>SC_REPORT_FATAL</code> .
9	Hex with no data lines	<code>SC_REPORT_FATAL</code> .
10	Hex overflow with non-zero word	<code>SC_REPORT_FATAL</code> .
11	Hex <code>0x / 0X</code> prefix	Parser strips the prefix.
12	Hex overflow with zero words	Tolerated.
13	<code>init_file_format = "garbage"</code>	Falls back to auto-detection.
14	<code>dbg_read32</code>	Happy / misaligned / OOB.
15	<code>ecc_enabled = false</code>	Injection is a no-op; reads never error.
16	<code>dbg_clear_ecc_errors</code>	Recovers a read after an uncorrectable injection.

5. Coverage goals & results

Goal: $\geq 95\%$ line and $\geq 80\%$ branch on `src/scratchpad_ram.cpp` , ASan-clean.

Achieved (LLVM source-based coverage, both benches merged):

Metric	<code>src/scratchpad_ram.cpp</code>
Function	100%
Line	98.5%
Region	93.6%
Branch	83.5%
ASan	0 errors

The few uncovered regions are defensive branches (e.g. the `TLM_COMMAND_ERROR_RESPONSE` arm for non-read/non-write commands is covered; residual misses are unreachable `std::min` edge folds).

6. Running the tests

```
./run_tests.sh           # Release build + run both benches
./run_tests.sh --ctest   # via ctest
./run_tests.sh --asan    # AddressSanitizer (+ LSan on Linux)
./run_tests.sh --coverage # LLVM coverage (Clang) or gcov (GCC)
```

7. Traceability to the specification

Spec section	Verified by
§8.1 Reads	Primary 1, 3, 6
§8.2 Writes / byte-enables	Primary 2, 4
§8.3 Preload	Primary 1; Negative 4–13
§8.4 / §9 Reset retention	Primary 5
§10 Bus interface / transport_dbg	Primary 7, 13
§11 Error handling	Primary 10; Negative 1–10
§12 SECDED ECC	Primary 9; Negative 15, 16
§4 CCI parameters	Primary 11

End of document.